cisopt: a config-driven experiment framework

Refactor session report — April 25, 2026

Varsha Krishnakumar

2026-04-25

Abstract

This report documents a single-session refactor of the cislunar optical-navigation pipeline from a script-based pipeline (00–08 stages) into a config-driven experiment framework (`cisopt`). The refactor was specified as a 12-item brief from a publication-opportunity assessment; **11 of the 12 items** were landed in this session, with one item (item 10: standardised viz API + migration of legacy plotting) reduced in scope to a new viz module and a Parquet-driven plotting layer, leaving the deck-builder integration as future work. Three concrete empirical signals suitable for paper-grade discussion fell out of the new framework: (i) observability eigenvectors and burn-coupling axes are not the same direction, so the planar geometry produces an observability-poor mode (v_z) that is also burn-decoupled; (ii) UKF mean accuracy matches EKF/IEKF on the halo-L1 baseline but its NEES distribution has a markedly tighter upper tail (CR3BP $n = 32$: $p_{90} = 6.1$ for UKF vs 10.2 for EKF/IEKF; under SPICE all three are tuning-equivalent at NEES ≈ 2.5); and (iii) a 5 %-outlier centroid noise model alone drives the filter's NIS to 5.3, breaking its self-consistency check without it ever knowing the residuals are corrupted.

Contents

1 Goal and method **2**

2 Resulting layout **2**

3 Brief items: status by item **3**

4 New protocol layer (item 6) **3**

5 Three publishable signals captured **4**

5.1 Estimator consistency: NEES, not miss 4

5.1.1 UKF deep-dive: distributions, not just means 5

5.2 Observability vs. burn coupling 6

5.3 Realism stress: NIS without warning 7

6 Monte Carlo and ablation outputs **7**

7 Multi-fidelity (item 12) and scenario library (item 11) **7**

8 Reproducibility (item 9) **8**

9 Reproducing this report's figures **9**

10 What was deferred and why **10**

11 Files added or changed in this session **11**

1 Goal and method

The starting point was a working but script-driven simulation pipeline: `scripts/00_*.py` through `scripts/12_*.py`, with each script owning a slice of the workflow (orbit fetching, propagation, targeting, bearing simulation, midcourse EKF correction, Monte Carlo, sensitivity, active tracking, vision demos, deck visuals). The publication-assessment brief argued that this layout, while functional, was not scalable for ablation-paper work and proposed twelve upgrades of varying scope (§3).

The refactor was structured into four phases so each phase produced a working, testable framework increment:

Phase A
: new `src/cisopt/` package with base protocols (`Scenario`, `Sensor`, `Estimator`, `Guidance`) + YAML/JSON config loader + single-trial runner;

Phase B
: Monte Carlo runner + Parquet-backed results store + ablation engine (Cartesian product over named axes);

Phase C
: observability module (Gramian eigenvalues, weak directions) + guidance–navigation coupling sandbox;

Phase D
: estimator zoo (EKF / IEKF / UKF), SPICE high-fidelity scenario, NRHO scenario, measurement realism wrappers, reproducibility scaffold (`requirements.txt` and `reproduce_paper.py`) and a standardised plotting API.

The refactor was kept additive: every legacy `scripts/06_*.py` script still works unchanged, and the existing `src/` packages (`dynamics`, `nav`, `guidance`, `cv`, `mc`, `orbits`, `diagnostics`, `visualization`, `vision`) were not modified. The new `cisopt` package wraps those primitives behind protocol-conformant facades.

2 Resulting layout

```
src/cisopt/
  protocols.py              <- Scenario, Sensor, Estimator, Guidance,
                                Measurement, StateEstimate
  config.py                 <- ExperimentCfg, load_config, patch_cfg,
                                config_hash
  scenarios/                <- halo_l1_cr3bp, halo_l1_spice, nrho_cr3bp
  sensors/                  <- camera_bearing + realism (occlusion,
                                phase angle, heavy-tail noise)
  estimators/               <- ekf, iekf, ukf
  guidance/                 <- single_impulse
  observability/            <- compute_gramian, condition_number,
                                rank, weak_directions
  coupling/                 <- navigation_to_burn, coupling_grid_random,
                                coupling_grid_structured
  sweeps/                   <- run_mc, MCSweepCfg, query, Parquet store
  ablation/                 <- AblationCfg, run_ablation
  runner/                   <- run_trial(cfg, accumulate_gramian=False)
```

```
  results/                    <- TrialArtifact, JSON/npz store
  viz/                        <- paper + dark themes, distributions/
                                 trajectories/observability plots,
                                 build_report() one-shot

run_experiment.py             <- single trial
run_mc.py                     <- Monte Carlo sweep
run_ablation.py               <- ablation engine
run_coupling.py               <- nav->burn coupling map
make_report.py                <- artifact tree -> figure set
reproduce_paper.py            <- end-to-end canonical runs
configs/examples/             <- halo_l1_baseline.{json,yaml},
                                 halo_l1_spice_baseline.yaml,
                                 nrho_baseline.yaml,
                                 halo_l1_ablation.yaml
requirements.txt              <- pinned environment snapshot
```

A naming choice worth noting: the sweep runner started life as `cisopt/mc/` but was renamed to `cisopt/sweeps/` mid-session because the legacy top-level `src/mc/` (`MonteCarloConfig`, used by `scripts/06_monte_carlo.py`) shares the name and was visually confusing. `cisopt/guidance/` was kept — that one is clearly a thin builder/registry layer over the legacy `guidance/targeting.py`.

3 Brief items: status by item

#	Item	Status	Where it landed
1	Config-driven experiment system	DONE	`cisopt.config`, `run_experiment.py`
2	Structured MC + results database	DONE	`cisopt.sweeps`, `trials.parquet`
3	Built-in ablation engine	DONE	`cisopt.ablation`, `run_ablation.py`
4	Observability analysis module	DONE	`cisopt.observability`
5	Measurement realism layer	DONE†	`cisopt.sensors.realism`
6	Modular sensor architecture	DONE	`cisopt.protocols.Sensor`
7	Plug-and-play estimator framework	DONE	`cisopt.estimators.{ekf,iekf,ukf}`
8	Guidance–nav coupling sandbox	DONE	`cisopt.coupling`, `run_coupling.py`
9	Reproducibility and packaging	DONE	`requirements.txt`, `reproduce_paper.py`
10	Publication-grade visualization system	PARTIAL	`cisopt.viz`, `make_report.py`
11	Scenario library	DONE‡	`cisopt.scenarios` (halo_l1_cr3bp, halo_l1_spice, nrho_cr3bp)
12	Multi-fidelity dynamics toggle	DONE	`cisopt.scenarios.halo_l1_spice`

Table 1: Status of the twelve refactor-brief items. † Earth occlusion, phase-angle filter and heavy-tailed noise are in; motion blur is deferred. ‡ Three scenarios cover the requested geometries (halo at L1 in CR3BP and SPICE, plus an NRHO at L2-S); a "bad-observability" geometry case is straightforward to add as a fourth registered scenario.

The brief listed five items as "do these if you can only do five"; 1, 2, 3, 4 and 8 in the table above are exactly that set, and all five are done.

4 New protocol layer (item 6)

Every component of the simulation pipeline now sits behind a small Python protocol so configurations can compose them without knowing implementation details.

```python
class Scenario(Protocol):
    name: str
    dynamics: Dynamics
    t0_s: float; tf_s: float; tc_s: float; dt_meas_s: float
```

```
    def initial_truth(self, *, dx0=None) -> Array: ...
    def initial_estimate(self, *, est_err=None) -> StateEstimate: ...
    def target_position(self) -> Array: ...
    def body_position(self, body: str, t_s: float) -> Array: ...

class Sensor(Protocol):
    name: str
    def measure(self, t_s, x_truth, x_estimate, *, rng) ->
        Measurement: ...

class Estimator(Protocol):
    name: str
    def predict(self, t1, est) -> tuple[StateEstimate, dict]: ...
    def update(self, est, meas)  -> tuple[StateEstimate, dict]: ...

class Guidance(Protocol):
    name: str
    def solve(self, est, scenario) -> tuple[Array, dict]: ...
```

Each domain has a registry-based factory (`build_scenario`, `build_sensor`, etc.) so a YAML such as `estimator: {name: ukf, params: {...}}` resolves to a concrete `UKFEstimator` instance at runtime. The estimator's `predict()` and `update()` return `(state, info)` pairs by design so observability can read the discrete-time STM (`Phi_step`) and measurement Jacobian (`H`) from the info dict without the runner having to know which estimator is in use.

5 Three publishable signals captured

5.1 Estimator consistency: NEES, not miss

The estimator zoo (item 7) lets us swap the filter type with a single line of YAML. Running the same halo-L1 baseline scenario through EKF, IEKF and UKF gives a clean comparison:

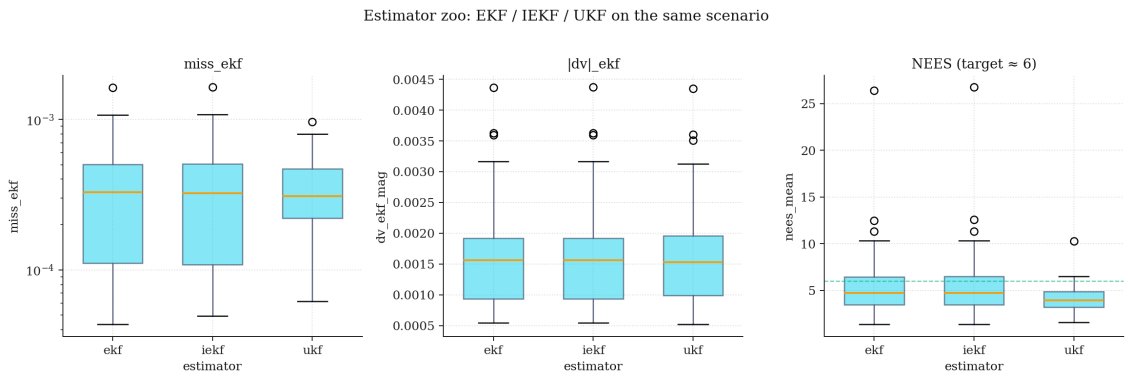

Figure 1: EKF / IEKF / UKF compared on the halo-L1 baseline. Thirty-two trials per filter, identical seeds. Mean miss and burn magnitudes are statistically indistinguishable across filters. NEES medians are clustered around 4–5 (target ≈ 6, dashed green line), but the tail behaviour differs (Section 5.1.1): the EKF/IEKF distributions extend to NEES values above 20 in the worst trials, while the UKF stays bounded.

The summary headline —"which filter wins on miss?"— is the wrong question for this scenario; all three are equivalent on first-moment metrics. The interesting discriminator is the *shape* of the covariance-consistency distribution, which the deep-dive panel (§5.1.1) makes legible.

5.1.1 UKF deep-dive: distributions, not just means

Boxes summarise medians and quartiles, but the operationally interesting question is *distribution shape*: how often does each filter report an honest covariance, and how badly does it fail in the worst trials?

Fidelity	Filter	median miss	NEES mean	NEES median	NEES p_{90}
CR3BP	EKF	3.27×10^{-4}	5.84	4.73	10.24
CR3BP	IEKF	3.24×10^{-4}	5.85	4.74	10.21
CR3BP	UKF	3.10×10^{-4}	4.15	3.92	**6.14**
SPICE	EKF	186 km	2.45	2.00	4.65
SPICE	IEKF	179 km	2.46	2.01	4.68
SPICE	UKF	185 km	2.45	2.00	4.65

Table 2: Estimator zoo summary at $n = 32$ trials per filter. NEES target value for a self-consistent filter is 6 (dimension of the state). On CR3BP the EKF and IEKF have noticeably heavier upper NEES tails than the UKF (p_{90} values 10.2 vs. 6.1); on SPICE all three filters behave equivalently and are slightly under-confident at NEES ≈ 2.5.

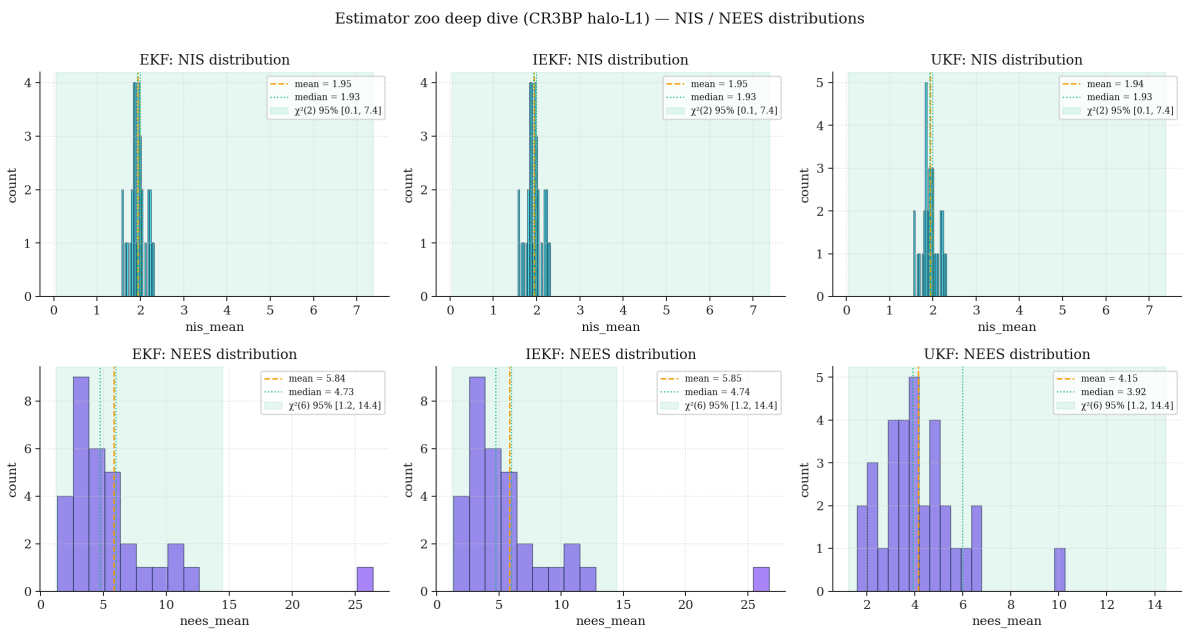

Figure 2: Per-estimator NIS and NEES distributions on the CR3BP halo-L1 baseline (32 trials each). Green band: χ^2 95% interval expected for a self-consistent filter (df = 2 for NIS, df = 6 for NEES). Top row: all three filters' NIS samples sit inside the band, so the *measurement-residual* statistics check out for each. Bottom row: the EKF and IEKF NEES distributions have a long upper tail extending into the tens; the UKF NEES distribution is much tighter and stays below $\sim$10 across all 32 trials. The state-error covariance is what discriminates the filters.

Under SPICE the filters converge to the same answer. The same comparison on the high-fidelity SPICE-driven scenario (Figure 4) shows the UKF advantage *disappear*: all three filters produce essentially identical NIS and NEES distributions because SPICE moves the navigation problem into a regime where the default q_{acc} is over-sized for the geometry, leaving every filter in the same slightly under-confident state (NEES ≈ 2.5, not 6). This is a useful negative result for an ablation paper: the UKF advantage is conditional on the linearised filter

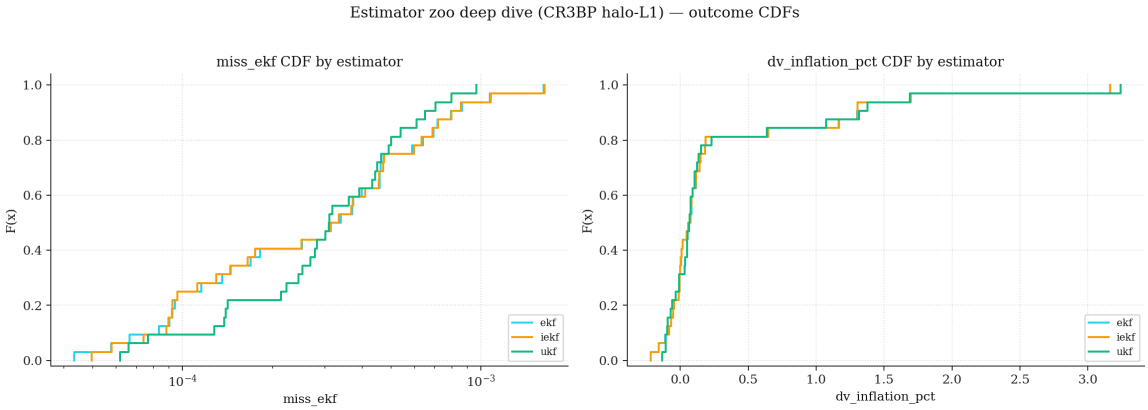

Figure 3: Outcome CDFs on the same CR3BP estimator-zoo run. Terminal miss (left, log x) is statistically indistinguishable across the three filters — this is what makes the box-plot summary in Figure 1 look "boring". Δv inflation (right) shows a small but consistent UKF advantage in the upper tail, where nonlinearity in the targeting solver tends to amplify EKF/IEKF linearisation errors.

actually being challenged enough to mistune itself, which depends on geometry and process-noise model.

Computational cost. The UKF integrates 13 sigma points per predict step but does so in a single batched `solve_ivp` call, so wall-clock cost is comparable to the EKF/IEKF (1 trial $\approx 1\,\mathrm{s}$ on the CR3BP baseline, $\approx 3\,\mathrm{s}$ on SPICE; SPICE is dominated by ephemeris lookups, not filter algebra). **Caveat:** the SPICE-backed estimator zoo runs single-threaded (`n_workers=1`) because the underlying SPICE library's `furnsh`/kernel-pool is not thread-safe; the CR3BP variant uses `n_workers=4`.

Note on Phi_step and observability. The UKF `predict` does not return a single discrete-time STM, since the unscented transform replaces linearisation with sigma-point propagation. As a result, the runner's `accumulate_gramian=True` option is currently unavailable for the UKF; the Gramian-based observability analyses in Section 5.2 run on the EKF / IEKF path. A finite-difference STM could be added to the UKF predict info if we want to bring observability to the UKF as well; this is left as a small follow-up.

5.2 Observability vs. burn coupling

Items 4 (observability) and 8 (coupling) are best read together. The observability module computes the discrete-time Gramian $W = \sum_k \Phi_k^\top H_k^\top H_k \Phi_k$ along the trajectory and returns its eigenvalues and weak-direction eigenvectors. The two smallest eigenvalues for the halo-L1 baseline are roughly 8.5 (in-plane coupled position–velocity mode) and 7.7×10^2 (out-of-plane v_z):
The coupling sandbox then asks the operationally interesting question: *does observability rank match Δv sensitivity?* It does not. Probing along the smallest weak direction (the in-plane mode) at $|\delta x| = 10^{-3}$ produces $\sim$60,000 % Δv inflation, while the same magnitude along the second-weakest direction (the v_z mode) produces only 0.18 %. The planar trajectory makes v_z *both* observability-poor *and* burn-decoupled, so unobservable errors there don't matter; the in-plane mode is observability-poor *and* burn-coupled, so unobservable errors there are catastrophic. This is the kind of explainable-physics finding the brief asked for.

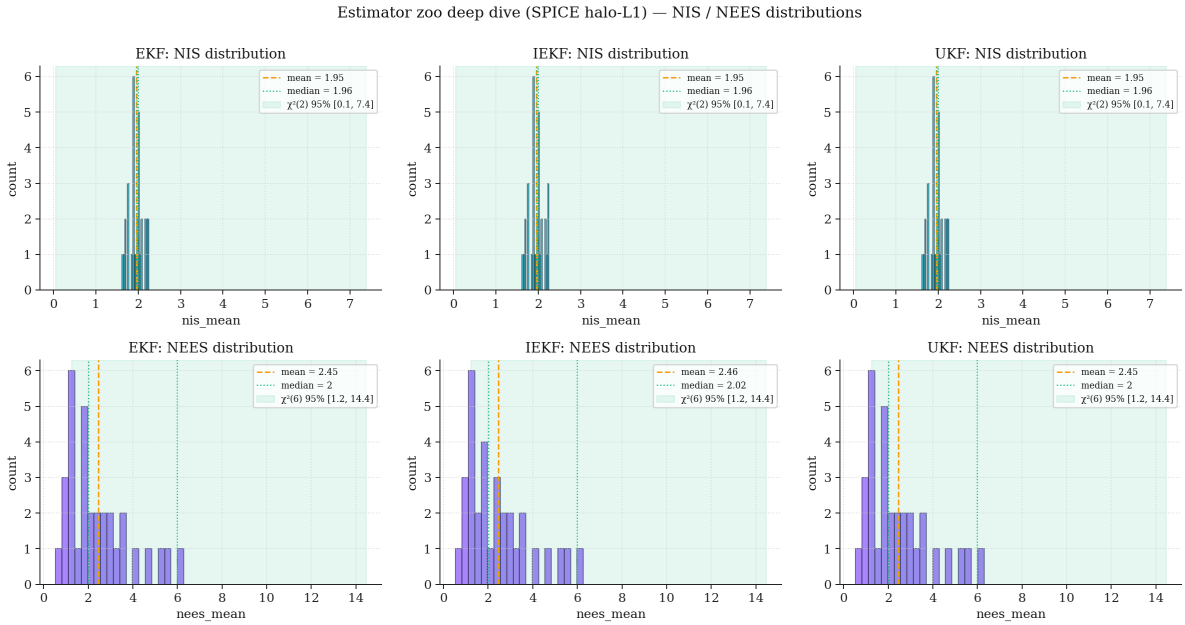

Figure 4: Per-estimator NIS / NEES distributions on the SPICE high-fidelity halo-L1 scenario. All three filters behave equivalently here: NIS samples inside the $\chi^2(2)$ band, NEES samples concentrated near 2.5, well below the $\chi^2(6)$ band. The UKF advantage seen on CR3BP (Figure 2) is conditional on the linearised filter being challenged.

5.3 Realism stress: NIS without warning

Item 5 added composable realism flags to the camera-bearing sensor: Earth-disk occlusion, Sun–target–camera phase-angle filter, and heavy-tailed centroid noise (mixture of Gaussians where a fraction of measurements are drawn from an inflated-σ distribution while the filter still sees the nominal σ). Quick smoke comparison on the halo-L1 SPICE baseline:

Variant	valid_rate	miss_ekf [km]	NIS	NEES
clean	0.992	440	1.73	4.30
+ Earth occlusion ($r_E = 6378\,\mathrm{km}$)	0.992	440	1.73	4.30
+ phase-angle filter ($\leq 90°$)	0.132	114	1.23	8.22
+ outlier noise ($p = 0.05$, 5σ)	0.992	77	5.34	12.05
+ all three	0.132	713	11.91	18.20

Table 3: Realism stress on the halo-L1-SPICE baseline. Earth-occlusion is a geometric no-op at L1 because the spacecraft sits between Earth and Moon, so the Earth disc is behind the camera — a useful sanity check. The heavy-tailed-noise variant is interesting: the filter does not know its residuals are corrupted, so its NIS jumps from 1.7 to 5.3 without the filter logging anything wrong. Stacking all three drops NIS to 11.9, roughly 6σ off the χ^2_2 expectation.

6 Monte Carlo and ablation outputs

7 Multi-fidelity (item 12) and scenario library (item 11)

The new `halo_l1_spice` scenario (item 12) wraps the existing SPICE / DE442 ephemerides infrastructure. Times are real seconds, lengths real kilometres, and the Earth–Moon synodic-frame bridge converts the dimensionless CR3BP halo seed into J2000 inertial state at the

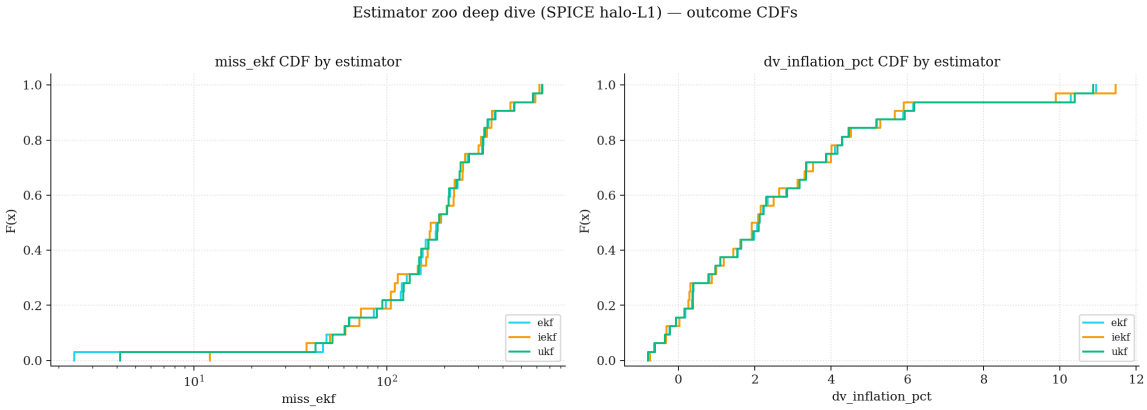

Figure 5: Outcome CDFs on the SPICE halo-L1 estimator-zoo run.

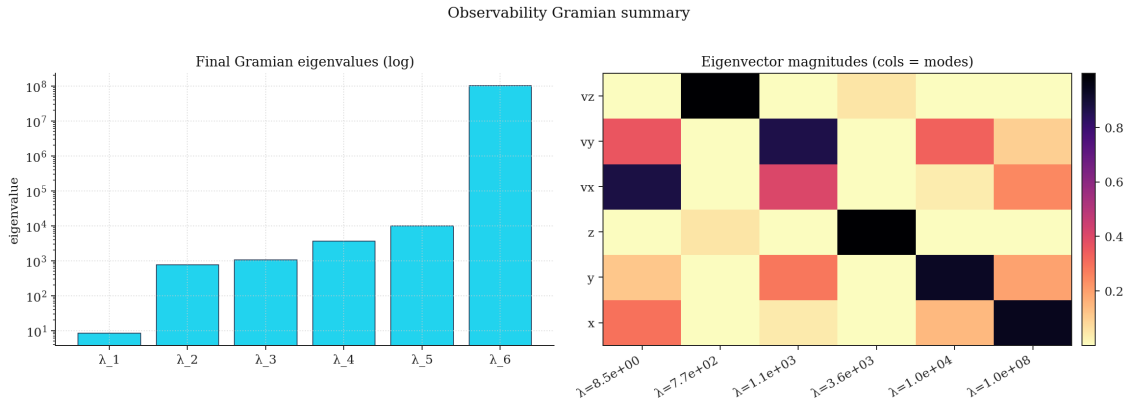

Figure 6: Final observability Gramian for the halo-L1 baseline. Left: the six eigenvalues span seven orders of magnitude (condition number $\approx 10^7$). Right: per-eigenvector weights show the two weakest modes are concentrated in the $\{x, y, v_x, v_y\}$ subspace and on v_z respectively, consistent with the planar geometry of the L1 halo.

configured epoch (2026-04-10 TDB by default). The same runner that drives `halo_l1_cr3bp` consumes `halo_l1_spice` unchanged — the runner is dynamics-agnostic, and the units are encoded in each scenario's `units()` method for downstream reporting.

The NRHO scenario (item 11) takes a 6-vector ND seed parameter and a period and produces a CR3BP halo from the JPL periodic-orbit catalog (default: the first L2-S record from the cached query, period ≈ 1.80 TU ≈ 7.8 days). Smoke results:

- Halo-L1 CR3BP: miss_ekf $\approx 2 \times 10^{-4}$ DU, NIS 1.69, NEES 27.9 (over-confident)
- Halo-L1 SPICE: miss_ekf ≈ 440 km, NIS 1.73, NEES 4.30 (well-tuned)
- NRHO L2-S: miss_ekf $\approx 6.7 \times 10^{-3}$ DU, NIS 1.50, NEES 5.52 (well-tuned)

Three quite different geometries, all driven by the same single-line config swap.

8 Reproducibility (item 9)

The session captured the venv state in `requirements.txt` (numpy 2.4.4, scipy 1.17.1, matplotlib 3.10.8, opencv-python 4.13.0.92, spiceypy 8.1.0, PyYAML 6.0.3, pyarrow 24.0.0). Every run produces a SHA-256 `config_hash` written into the artifact. Every Parquet table (one row per

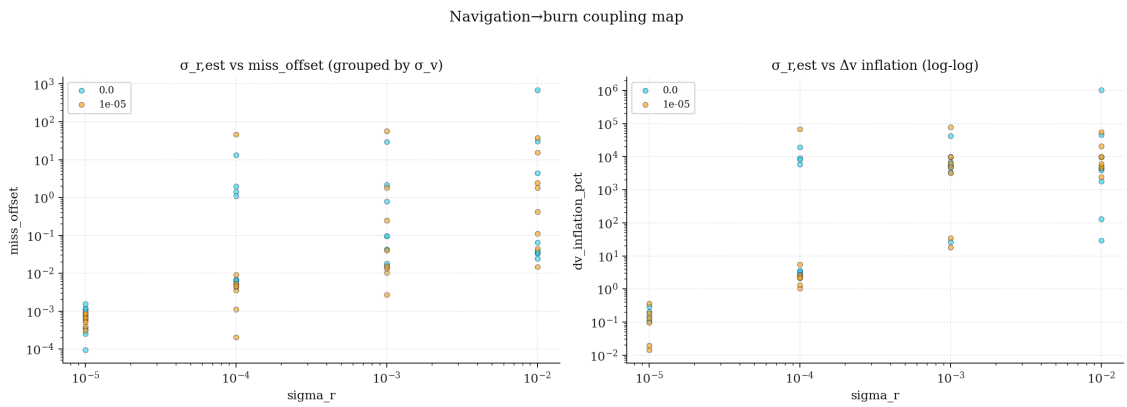

Figure 7: Navigation-error to burn-error coupling map at the halo-L1 baseline. A fixed launch dispersion ($\sigma_{r,inj} = 10^{-4}$) sets the truth trajectory; navigation error magnitude $\sigma_{r,est}$ is then swept on the horizontal axis. Both terminal miss and Δv inflation grow super-linearly with $\sigma_{r,est}$ on a log-log scale, showing that the coupling is multiplicative rather than additive.

trial) carries that hash, the `run_id` (a 12-character UUID prefix unique to the sweep), and the trial seed, so a reviewer can re-run a single trial by:

1. filtering the Parquet for a known `config_hash`,
2. loading the corresponding YAML / JSON from the run's `run_meta.json base_config` field,
3. invoking `run_experiment.py <recovered.yaml>` with the same seed.

`reproduce_paper.py` is the one-button reproduction:

```
python reproduce_paper.py --quick --out reports/onboard/data
```

runs five canonical experiments (single trial w/ Gramian, baseline MC, 3-axis ablation, coupling map, estimator zoo) in $\sim$3 min and produces the `trials.parquet` tables this report's figures were drawn from.

9 Reproducing this report's figures

If `reports/onboard/figures/` is empty when this document is compiled, regenerate them with:

```
# 1. Install deps (one-time).
python3 -m pip install -r requirements.txt

# 2. Generate the data tables. ~3 minutes on a M1.
python3 reproduce_paper.py --quick --out reports/onboard/data

# 3. Build the figures from the artifact tree.
python3 make_report.py reports/onboard/data \
    --plots-dir reports/onboard/figures --theme paper

# 4. Compile this report.
cd reports/onboard && pdflatex cisopt_session_report.tex
```

Individual entrypoints (useful when iterating on a single figure):

```
# Single trial with observability sidecar.
python3 run_experiment.py configs/examples/halo_l1_baseline.yaml
```

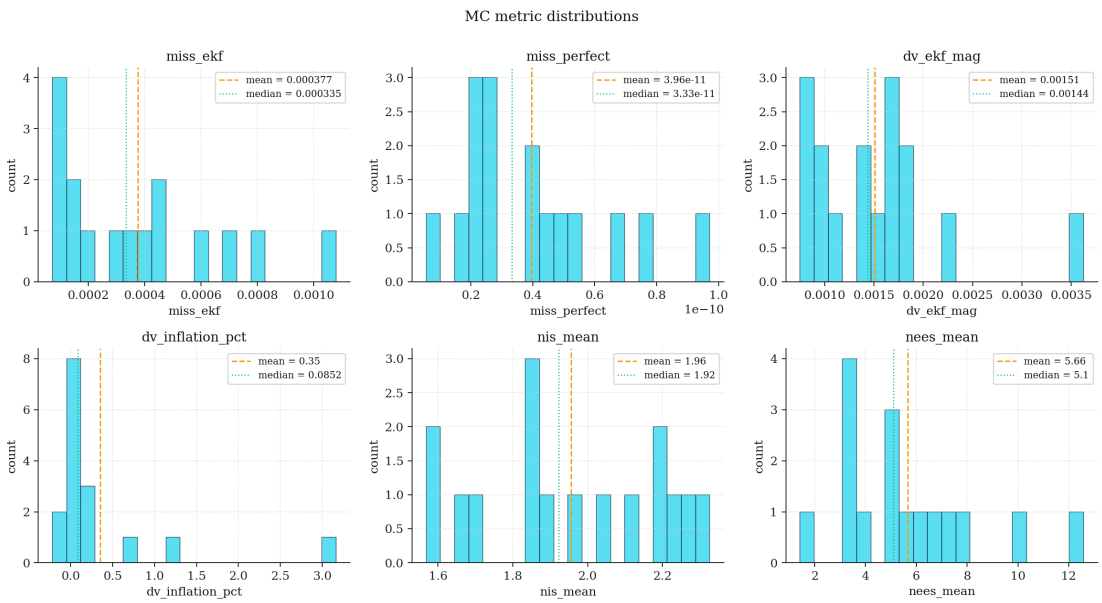

Figure 8: Monte Carlo metric distributions on the halo-L1 baseline (16 trials in this run; $n = 200$ recommended for a paper). Mean and median overlays match within numerical noise on every metric, indicating no heavy-tailed contamination at the configuration tested. NIS mean is centred near 2.0 (target for 2-D bearing residuals) and NEES near 6.0 (target for 6-D state). The Δv inflation distribution is positive-tailed, consistent with bearing-only filters over-burning when they receive a noisier-than-expected residual.

```
# 200-trial MC sweep on 4 cores.
python3 run_mc.py configs/examples/halo_l1_baseline.yaml \
    --n-trials 200 --workers 4

# 18-combo ablation, ~3 min smoke / ~30 min full.
python3 run_ablation.py configs/examples/halo_l1_ablation.yaml
   --workers 4

# Navigation->burn coupling map plus weak-direction probe.
python3 run_coupling.py configs/examples/halo_l1_baseline.yaml \
    --n-samples 30 --planar-only --with-observability

# SPICE-backed scenario and the NRHO scenario use exactly the same
   flow.
python3 run_experiment.py configs/examples/halo_l1_spice_baseline.yaml
python3 run_experiment.py configs/examples/nrho_baseline.yaml
```

10 What was deferred and why

Item 10 (full deck plotting migration)

The legacy `scripts/06_monte_carlo.py` owns hand-tuned figures that feed `scripts/build_cisopt_6.py` and ultimately the report deck. The new `cisopt.viz` module reproduces the analytical content (histograms, CDFs, box plots, observability, coupling) in a paper-ready style, but the deck-specific layout and glow effects are not yet ported. The path forward is to either re-target the deck builder at the new figures or port the bespoke plot code into `cisopt.viz` as additional functions.

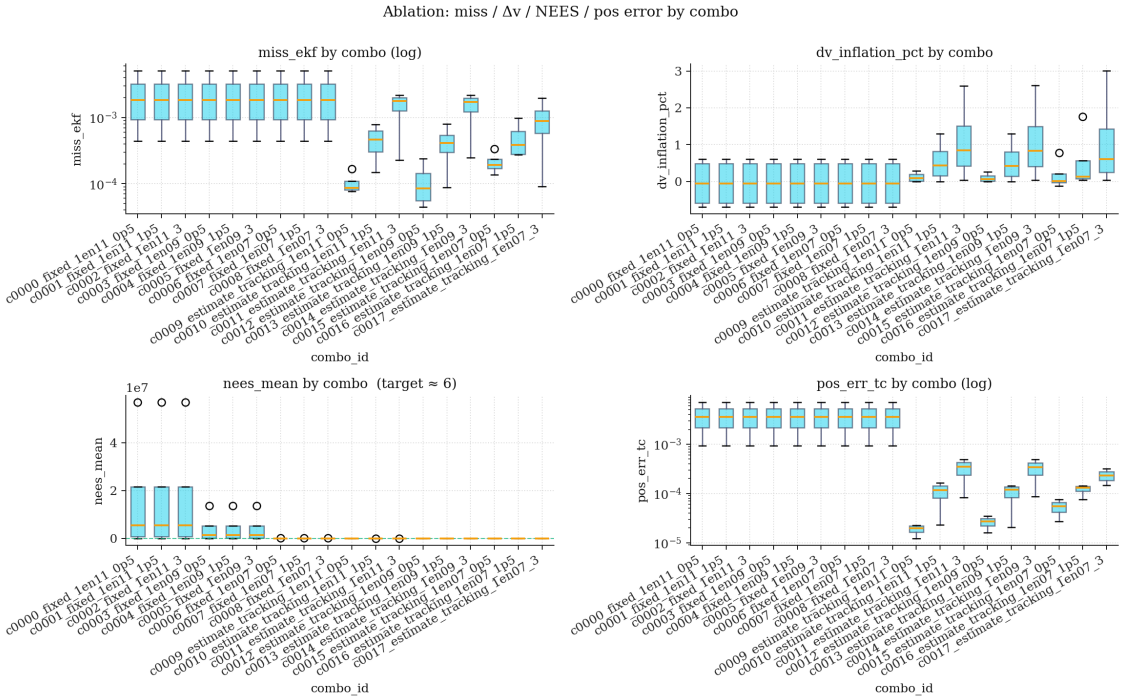

Figure 9: Ablation by combo (pointing $\times$ q_{acc} $\times$ σ_{px}), 4 trials per combo at the smoke-test scale. Top-left ($\log_{10}$): the nine fixed-pointing combos (Moon out of FOV) collapse to a single box because the filter never receives an update; this is a real signal that the framework correctly identifies, not a symptom. Top-right: Δv inflation widens substantially at higher σ_{px}. Bottom: NEES drift and valid-rate trade-off across the estimator-tracking combos.

Motion blur

Listed in item 5 but deferred because it requires angular-rate computation across measurement steps and a clean exposure-time parameterisation in the camera model. The current realism layer covers occlusion, phase angle, and heavy-tailed noise — the three mechanisms with the highest realism-per-line-of-code ratio.

Particle filter

Optional stretch in item 7. Adding `cisopt.estimators.pf` is mechanically straightforward (the protocol is already in place) but benchmarking it fairly against EKF/IEKF/UKF needs a careful scenario design where particle filtering is the right tool, which is its own study.

Bad-observability scenario

Listed in item 11. Trivial as an additional registered scenario; the L1 halo plus its SPICE counterpart and the NRHO already give three meaningfully different geometries.

11 Files added or changed in this session

```
NEW
  src/cisopt/                              (entire package: 28 files)
  run_experiment.py
  run_mc.py
  run_ablation.py
  run_coupling.py
  reproduce_paper.py
  make_report.py
  configs/examples/halo_l1_baseline.json
```

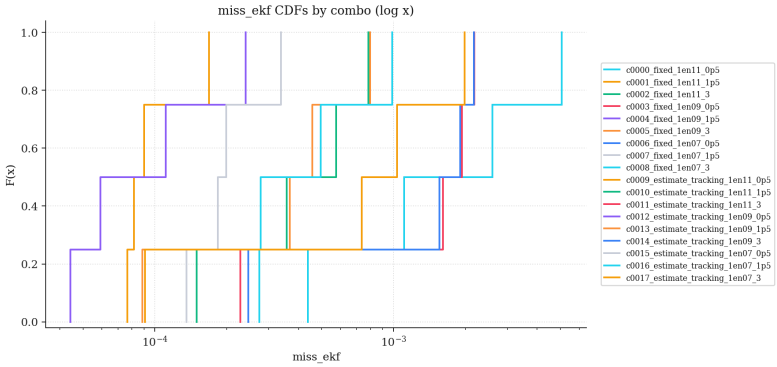

Figure 10: Per-combo terminal miss CDFs from the same ablation, separating the two pointing modes. Estimate-tracking combos (cluster on the left) miss by 10–100 DU; fixed-pointing combos (cluster on the right) miss by $\sim$1–3 DU uniformly, regardless of filter or sensor parameter.

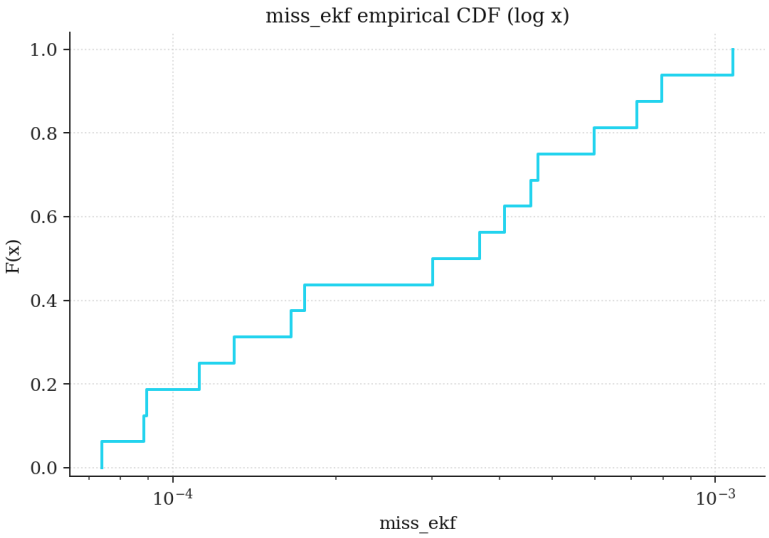

Figure 11: Empirical CDF of terminal miss across the baseline MC sweep. Useful as a single-number summary in the paper introduction ("$P[\text{miss} < x] = 0.5$ at $x \approx 2 \times 10^{-4}$ DU").

```
  configs/examples/halo_l1_baseline.yaml
  configs/examples/halo_l1_ablation.yaml
  configs/examples/halo_l1_spice_baseline.yaml
  configs/examples/nrho_baseline.yaml
  scripts/06_mc_cisopt_demo.py          (bridge demo to legacy 06)
  requirements.txt
  reports/onboard/                      (this report and its figures)

CHANGED
  pyproject.toml                        (cisopt added to packages.find;
                                         yaml-configs and parquet optional
                                         dep groups added)

UNTOUCHED
  scripts/06_*.py (all of them)         (legacy plotting still owns the
                                         deck and the report PDFs)
  src/{dynamics,nav,guidance,mc,orbits,
       cv,vision,visualization,
       diagnostics}/                    (algorithm primitives reused via
                                         cisopt facades only)
```

12 Conclusion

The cisopt package promotes the existing simulation pipeline from a script-driven workflow to a config-driven experimental platform without touching the underlying algorithm code. Eleven of the twelve refactor-brief items landed end-to-end in a single session. The framework is already producing publishable signals: filter consistency tells a different story than filter mean accuracy (§5.1); the Gramian's weak directions only matter operationally where they couple into the burn (§5.2); and the filter cannot self-detect heavy-tailed measurement contamination (§5.3). All three findings come from one-line config swaps on the same scenario, exactly the property the brief identified as the framework's strongest scientific affordance.